

MEMORIAL TÉCNICO DE DESENVOLVIMENTO

Portal de Solicitações Internas

Documentação técnica do projeto, arquitetura, funcionalidades, segurança e execução.

Desenvolvedor: Marcos Antônio Leite da Silva

Repositório: MarcosPholker/Portal-De-Solicitacoes-Internas

2026

1. Identificação e objetivo

O **Portal de Solicitações Internas** é uma aplicação web destinada ao gerenciamento de solicitações internas de uma organização. A solução permite autenticação de usuários, abertura e acompanhamento de solicitações, alteração de status, consultas, filtros e visualização de informações consolidadas.

Este memorial registra as principais características técnicas do projeto, sua organização, tecnologias utilizadas, mecanismos de segurança, persistência de dados, comunicação entre frontend e backend e execução por Docker.

2. Arquitetura

A aplicação foi organizada em três componentes principais: frontend, backend e banco de dados PostgreSQL. O frontend é responsável pela interface e interação com o usuário; o backend concentra a API REST, regras de negócio, autenticação e autorização; o PostgreSQL realiza a persistência.

Camada	Tecnologia	Responsabilidade
Frontend	React + Vite	Interface, navegação e consumo da API
Backend	Java + Spring Boot	API REST, regras de negócio e segurança
Banco	PostgreSQL + JPA/Hibernate	Persistência e consultas
Infraestrutura	Docker Compose	Orquestração dos serviços

3. Tecnologias utilizadas

Backend: Java 21, Spring Boot 4.1.1, Spring Data JPA, Spring Security, Bean Validation, Maven, PostgreSQL e JWT 0.12.6.

Frontend: React 19, React DOM 19, Vite 8, React Router DOM, Axios, JavaScript e CSS.

Infraestrutura: Docker, Docker Compose, PostgreSQL 18, Node 22 Alpine para build do frontend e Nginx Alpine para servir a aplicação.

As configurações sensíveis são fornecidas por variáveis de ambiente, evitando manter credenciais diretamente no código-fonte.

4. Organização do backend

O backend utiliza separação por responsabilidades, com controllers, DTOs, entities, enums, repositories, services, segurança e tratamento de exceções.

Controllers: recebem e encaminham as requisições HTTP.

DTOs: controlam os dados de entrada e saída da API.

Entities: representam os dados persistidos.

Repositories: realizam o acesso aos dados através do Spring Data JPA.

Services: concentram as regras de negócio.

Security: contém autenticação JWT, filtro de segurança e handlers de autenticação/autorização.

Exceptions: padronizam o tratamento de situações de erro.

5. Funcionalidades implementadas

- Cadastro e autenticação de usuários.
- Login com geração de token JWT.
- Controle de acesso a recursos protegidos.

- Criação de solicitações internas.
- Consulta de solicitações.
- Consulta das solicitações do usuário autenticado.
- Consulta de detalhes de uma solicitação.
- Atualização e exclusão de solicitações.
- Alteração e acompanhamento de status.
- Filtros por título, categoria, status e período.
- Dashboard com informações consolidadas.
- Validação dos dados recebidos pela API.
- Tratamento global de exceções.

6. Autenticação e segurança

A autenticação utiliza JWT (JSON Web Token). Após o login, o backend gera um token que deve ser enviado nas requisições protegidas por meio do cabeçalho HTTP **Authorization**, no formato **Bearer <token>**.

O Spring Security controla os recursos protegidos. O filtro JWT valida o token recebido e estabelece o contexto de autenticação quando a requisição possui credenciais válidas.

As senhas são armazenadas utilizando codificação segura por meio do mecanismo de senha do Spring Security, não sendo mantidas em texto puro.

Foram implementados tratamentos específicos para credenciais inválidas, usuário inexistente, usuário não autenticado e acesso negado.

7. Principais endpoints

Método	Endpoint	Função
POST	/auth/register	Cadastro de usuário
POST	/auth/login	Login e geração do JWT
POST	/internalrequest/create	Criação de solicitação
GET	/internalrequest	Consulta e filtros
GET	/internalrequest/mine	Solicitações do usuário
GET	/internalrequest/dashboard	Dados do dashboard
GET	/internalrequest/{id}	Detalhes
PUT	/internalrequest/update/{id}	Atualização
DELETE	/internalrequest/delete/{id}	Exclusão

8. Categorias e status

As solicitações são classificadas por categorias representadas no backend por enumerações, incluindo TI, RH, área comercial, financeiro e infraestrutura.

O ciclo de atendimento utiliza os estados **OPEN**, **IN_PROGRESS** e **COMPLETED**, permitindo acompanhar a evolução das solicitações.

9. Persistência e banco de dados

O banco utilizado é o PostgreSQL, com o banco **portal_solicitacoes**. O acesso é realizado através do Spring Data JPA e Hibernate.

As entidades representam os objetos principais do domínio, enquanto repositories encapsulam as operações de persistência e consultas utilizadas pelo sistema.

A configuração utiliza **spring.jpa.hibernate.ddl-auto=update**, permitindo que o Hibernate mantenha a estrutura compatível com as entidades durante a execução.

10. Frontend

O frontend foi desenvolvido em React e utiliza Vite para build e desenvolvimento. React Router realiza a navegação e Axios é utilizado para comunicação HTTP com o backend.

A aplicação possui telas relacionadas à autenticação, cadastro, criação de solicitações, consulta, detalhes e dashboard.

Nas operações protegidas, o frontend envia o token JWT no cabeçalho de autorização.

11. Docker e execução

O projeto utiliza Docker Compose para iniciar frontend, backend e PostgreSQL de forma integrada.

O PostgreSQL utiliza o volume persistente **postgres_data**. O backend é construído a partir de **backend/** e exposto na porta 8080. O frontend é construído a partir de **frontend/** e servido pelo Nginx na porta 3000.

Dentro da rede Docker, o backend acessa o PostgreSQL utilizando o nome do serviço **postgres** como host.

12. Estrutura do projeto

```
Portal-De-Solicitacoes-Internas/  
■■■ backend/  
■■■ database/  
■■■ docs/  
■■■ frontend/  
■■■ .env.example  
■■■ .gitignore  
■■■ docker-compose.yml  
■■■ README.md
```

13. Variáveis de ambiente

O ambiente utiliza principalmente:

- **DB_USERNAME** — usuário do PostgreSQL.
- **DB_PASSWORD** — senha do PostgreSQL.
- **JWT_SECRET** — chave usada para assinatura e validação dos tokens.

O arquivo **.env.example** funciona como modelo para configurar o ambiente local. Credenciais reais devem permanecer no arquivo **.env** local e não devem ser versionadas.

14. Comandos de execução

A partir da raiz do projeto, o ambiente Docker pode ser iniciado com:

```
docker compose up --build -d
```

Para verificar os containers:

```
docker compose ps
```

Para acompanhar os logs do backend:

```
docker compose logs -f backend
```

Acessos principais: frontend em **http://localhost:3000**, backend em **http://localhost:8080** e PostgreSQL em **localhost:5432**.

15. Validação e tratamento de erros

A API utiliza Bean Validation para validar dados recebidos e possui tratamento global de exceções. Foram criadas exceções específicas para cenários como e-mail já cadastrado, credenciais inválidas, usuário não encontrado, usuário não autenticado e solicitação inexistente.

Os códigos HTTP são utilizados para comunicar o resultado das operações, diferenciando respostas de sucesso, dados inválidos, ausência de autenticação, acesso negado e recursos não encontrados.

16. Testes automatizados

O projeto possui testes para componentes de serviço e controller, incluindo testes relacionados a usuários e solicitações internas. Os testes verificam regras de negócio e comportamentos relevantes da API.

A cobertura pode ser ampliada posteriormente para cenários adicionais de segurança, filtros, autenticação e integração.

17. Decisões técnicas

- Separação entre frontend e backend para facilitar manutenção e evolução.
- Uso de DTOs para controlar os dados expostos pela API.
- Uso de Services para concentrar regras de negócio.
- Uso de Spring Security e JWT para autenticação.
- Uso de PostgreSQL como banco relacional.
- Uso de Docker Compose para padronizar o ambiente.
- Uso de variáveis de ambiente para configurações sensíveis.
- Tratamento global de exceções para respostas consistentes.

18. Evidências da entrega

Recomenda-se manter em **docs/evidencias/** capturas de tela que comprovem o funcionamento da aplicação.

- Tela de login e autenticação.
- Tela de cadastro.
- Criação de solicitação.
- Lista de solicitações e filtros.
- Detalhes e alteração de status.
- Dashboard.
- Terminal com docker compose ps mostrando os serviços ativos.
- Requisições da API executadas com sucesso.
- Estrutura do projeto no editor.

Um vídeo curto também pode ser utilizado como evidência complementar, demonstrando o fluxo principal da aplicação.

19. Melhorias futuras

- Recuperação e alteração de senha.
- Perfis e papéis de acesso mais detalhados.
- Paginação nas consultas.
- Ampliação da cobertura de testes.
- Swagger/OpenAPI.
- Logs estruturados e observabilidade.
- Pipeline de CI/CD.
- Deploy em ambiente de nuvem.

20. Conclusão

O Portal de Solicitações Internas foi desenvolvido com separação entre frontend, backend e banco de dados, utilizando Java, Spring Boot, React e PostgreSQL. A solução contempla autenticação JWT, controle de acesso, gerenciamento de solicitações, filtros, dashboard, validação, tratamento de exceções e persistência relacional.

A utilização de Docker Compose permite reproduzir o ambiente de execução de maneira padronizada. A organização por responsabilidades no backend também facilita a manutenção e a evolução do sistema.

Este memorial complementa o código-fonte e registra as principais decisões técnicas e características da implementação.